Apache Kafka Task Queue System

Complete beginner documentation based on the actual repository

60 numbered topics: understanding words, exact project behavior, and workflows

Project facts

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Contents

No.	Topic
1	Project Overview
2	What Is Kafka?
3	Real-Life Analogy
4	Kafka Architecture
5	Complete Project Workflow
6	Kafka Producer
7	Producer Startup Flow
8	Producer Message Flow
9	Kafka Message Structure
10	Topic
11	Partitions
12	Message Key
13	Kafka Broker
14	Kafka Consumer
15	Consumer Startup Flow
16	Consumer Groups
17	Offset
18	Consumer Polling
19	Acknowledgement
20	Message Delivery
21	Serialization
22	Project Configuration and Environment Variables
23	Request/Response vs Kafka
24	Complete Message Lifecycle
25	Complete Code Execution Flow
26	Error Handling
27	Kafka Docker Setup
28	ZooKeeper vs KRaft
29	Kafka CLI Commands
30	Testing the Project
31	Debugging Guide
32	Kafka Consumer Lag
33	Beginner Project to Real Project
34	Real-World Use Cases
35	Scalability
36	Kafka Replication
37	Retention
38	Kafka vs Traditional Message Queues
39	Security
40	Monitoring
41	Kafka Design Best Practices
42	Complete API Flow

No.	Topic
43	Database Flow
44	Complete Architecture Diagram
45	Sequence Diagram
46	Folder Structure
47	File-by-File Explanation
48	Kafka Terminology Glossary
49	Interview Questions
50	2-Minute Project Explanation
51	5-Minute Technical Explanation
52	Common Beginner Confusions
53	Documentation Files
54	Master Markdown File
55	PDF Document
56	PDF Quality Check
57	Final Kafka Flow
58	Final Beginner Explanation
59	Final Project Report
60	Most Important Rule: One Message

1. Project Overview

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

This is an asynchronous task queue. A browser calls Flask, Flask creates a task record and publishes JSON to Kafka, and consumer.py processes it in the background. The dashboard reads local JSON state to show QUEUED, PROCESSING, or COMPLETED.

Workflow

User -> POST /submit-task -> producer.py -> task-queue -> consumer.py -> task_results.json -> GET /tasks -> dashboard

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

2. What Is Kafka?

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Kafka is a durable, distributed log. It lets the web request finish quickly while another process handles slow work later. In this project Kafka separates the Flask web server from the worker.

Workflow

Without Kafka: browser waits for work. With Kafka: browser submits an event, Kafka stores it, and the worker reads it when ready.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

3. Real-Life Analogy

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Think of an airport baggage system. The Flask producer is check-in, Kafka is the conveyor system, task-queue is a labeled belt, a partition is a lane, the consumer is a baggage worker, and an offset is the bag position. The analogy stops at delivery details: Kafka retains records and supports replay; luggage systems usually do not.

Workflow

Check in task -> label JSON -> conveyor stores it -> worker takes it -> worker records completion.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

4. Kafka Architecture

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The running topology is one Docker broker and one ZooKeeper service. The topic is task-queue. The consumer group has one worker process in the normal setup. Replication and multiple brokers are not configured for this beginner project.

Workflow

Flask producer -> broker -> task-queue partition -> task-processor-group -> consumer worker.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

5. Complete Project Workflow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

At startup, producer.py loads submitted_tasks.json and tries to create KafkaProducer. consumer.py creates KafkaConsumer with auto_offset_reset=earliest and enable_auto_commit=True. On a task request, the producer sends JSON; the worker processes it for 2, 3.5, or 5 seconds depending on priority.

Workflow

Start Docker -> start consumer -> start Flask -> submit task -> send -> consume -> process -> write result -> dashboard refresh.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

6. Kafka Producer

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

producer.py is the producer. get_kafka_producer() configures bootstrap_servers, JSON value_serializer, request timeout, and retries. submit_task() creates the task payload and calls producer.send('task-queue', value=task_payload).

Workflow

HTTP input -> validate -> create TASK-XXXXXXXX -> serialize -> send -> flush -> future.get metadata.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

7. Producer Startup Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The producer is created lazily and cached in producer_instance. Startup attempts two connections; a request can make a one-attempt connection. If the broker is still starting, NoBrokersAvailable is logged and the local fallback may be used.

Workflow

Create client -> connect to localhost:9092 -> keep client -> reuse for requests.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

8. Producer Message Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

producer.send() passes a Python dictionary to the JSON serializer. Kafka receives bytes, assigns a partition and offset, and future.get() returns metadata. The code logs topic, partition, and offset after flush and acknowledgement.

Workflow

Dictionary -> JSON UTF-8 bytes -> broker -> partition/offset -> producer metadata.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

9. Kafka Message Structure

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

A project record contains task_id, task_name, priority, status, submitted_at, completed_at, duration_seconds, and result_message. The Kafka key is not supplied, so the producer does not intentionally choose a partition by business key. Kafka metadata adds partition and offset outside the value.

Workflow

Value fields -> serialized JSON -> Kafka record -> worker deserializes to a dictionary.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

10. Topic

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

task-queue is the only application topic named in the code. The producer writes to it and the consumer subscribes to it. Multiple producers could publish to it; consumers in different groups could independently read it.

Workflow

Topic name is a constant in both producer.py and consumer.py.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

11. Partitions

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

A partition is an ordered append-only lane. This project does not explicitly create or configure partitions, so Kafka's default topic creation behavior applies. The Docker broker enables auto topic creation. Verify the real count with kafka-topics --describe.

Workflow

Record -> partition selection -> increasing offset within that partition.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

12. Message Key

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

No key argument is passed to producer.send(). Therefore task identity or priority is not used to guarantee same-partition ordering. If keyed ordering is needed, send key=task_id or another stable business key and plan partitions deliberately.

Workflow

No explicit key -> Kafka's default partitioner behavior -> chosen partition.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

13. Kafka Broker

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The broker is the Kafka server in the container named kafka-broker. Host applications use localhost:9092; Docker-internal clients would use kafka:29092. One broker is configured, with single-replica internal topics.

Workflow

Producer/consumer TCP connection -> broker log -> response or fetched records.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

14. Kafka Consumer

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

consumer.py is the worker. It creates KafkaConsumer with the topic, bootstrap server, group_id, JSON value_deserializer, earliest reset, and automatic commits. For each message it calls process_task().

Workflow

Broker record -> message.value -> process_task -> result file.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

15. Consumer Startup Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The consumer connects to localhost:9092, joins task-processor-group, and subscribes by passing task-queue to KafkaConsumer. The iterator over consumer continuously obtains records; consumer_timeout_ms=1000 lets the loop regain control when no record arrives.

Workflow

Create -> join group -> subscribe -> fetch/iterate -> process.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

16. Consumer Groups

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

A consumer group lets workers share partitions. In this project the group is task-processor-group. With one partition, only one member can actively own that partition at a time. Running another consumer with the same group shares work only when enough partitions exist.

Workflow

Group coordinator assigns partitions -> one member reads each assigned partition.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

17. Offset

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

An offset is the position of a record within a partition. consumer.py prints message.partition and message.offset. enable_auto_commit=True commits offsets automatically about every 1000 ms. A crash around processing and commit can cause reprocessing, so this is not exactly-once.

Workflow

Fetch offset N -> process -> auto commit later -> restart resumes from committed position.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

18. Consumer Polling

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The kafka-python consumer pulls records by iterating over KafkaConsumer. Kafka does not call process_task directly; the client fetches batches or records from the broker. This allows the consumer to control its pace and apply backpressure.

Workflow

Consumer fetch request -> broker returns available records -> iterator yields message.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

19. Acknowledgement

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The producer waits for metadata using flush(timeout=3) and future.get(timeout=3). No explicit acks setting is configured, so the client/library default applies. Do not describe the project as using a deliberate acks=all policy unless you configure it.

Workflow

Send -> flush -> future result -> success or exception.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

20. Message Delivery

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The intended normal path is at-least-once-like: a record can be processed and then seen again if the process fails before its automatic offset commit. Exactly-once is not implemented. At-most-once is not intentionally configured either.

Workflow

Delivery guarantee depends on commit timing and worker failure.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

21. Serialization

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Producer values use json.dumps(v).encode('utf-8'); consumer values use json.loads(m.decode('utf-8')). This is application-level JSON serialization, not Avro or a schema registry.

Workflow

Python dict -> JSON text -> UTF-8 bytes -> Kafka -> UTF-8 decode -> Python dict.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

22. Project Configuration and Environment Variables

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Configuration is read mostly from code and environment. producer.py and consumer.py use KAFKA_BOOTSTRAP_SERVERS, defaulting to localhost:9092. The topic and group are constants. Docker Compose defines ports, listeners, ZooKeeper, auto topic creation, and single-replica internal settings.

Workflow

Environment/defaults -> Kafka client configuration -> broker connection -> topic/group behavior.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

23. Request/Response vs Kafka

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

REST is synchronous request/response: the caller waits for a response. Kafka is asynchronous: the producer publishes an event and the consumer processes it later. This project uses REST for submission and status queries, Kafka for background task transport.

Workflow

POST submits intent; GET /tasks reads local status; Kafka carries work.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

24. Complete Message Lifecycle

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

One task starts as HTTP JSON, receives a generated ID and QUEUED status, is serialized, stored in task-queue, assigned partition/offset, fetched by the consumer, deserialized, marked PROCESSING, simulated, marked COMPLETED, and merged back into submitted_tasks.json for the dashboard.

Workflow

Created -> producer -> serializer -> broker/topic/partition/offset -> consumer -> processing -> result -> UI.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

25. Complete Code Execution Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The main functions are producer.py: load_persisted_tasks(), get_kafka_producer(), submit_task(); and consumer.py: create_kafka_consumer(), process_task(), main(). The actual call chain is Flask route -> producer.send -> Kafka -> consumer iterator -> process_task -> update_task_results.

Workflow

main/app.run -> route -> send; main -> consumer -> process_task.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

26. Error Handling

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Implemented handling includes producer retries during connection, send exceptions with local queue_buffer.json fallback, consumer reconnect attempts, atomic result-file replacement, and graceful Ctrl+C shutdown. Invalid task_name returns HTTP 400; invalid priority becomes MEDIUM. There is no dead-letter topic or retry topic.

Workflow

Failure -> log -> retry, local buffer, or task remains dependent on persisted state.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

27. Kafka Docker Setup

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

docker-compose.yml runs cp-zookeeper:7.5.0 on 2181 and cp-kafka:7.5.0 on 9092. It advertises localhost:9092 for host scripts and kafka:29092 internally, enables auto topic creation, and uses one broker. Volumes are not declared, so persistence is not explicitly configured.

Workflow

docker compose up -d -> ZooKeeper -> broker -> Python clients.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

28. ZooKeeper vs KRaft

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

This project uses ZooKeeper through KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181. It does not use KRaft. KRaft is Kafka's newer metadata mode; migration would require a different Compose configuration and is outside the current implementation.

Workflow

ZooKeeper coordinates broker metadata; broker serves records.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

29. Kafka CLI Commands

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Useful commands are: docker compose up -d; docker compose ps; docker exec -it kafka-broker kafka-topics --bootstrap-server localhost:9092 --list; --describe --topic task-queue; kafka-console-consumer --topic task-queue --from-beginning; docker compose down. Use the container's CLI so versions match.

Workflow

Start -> inspect -> create/describe -> produce/consume -> stop.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

30. Testing the Project

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Test path: start Compose, run consumer.py, run producer.py, open localhost:5001, submit a task, watch QUEUED become PROCESSING and COMPLETED, then call GET /tasks. The CLI also supports POST /submit-task and GET /health.

Workflow

Infrastructure -> worker -> API -> event -> result -> dashboard.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

31. Debugging Guide

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

For connection errors, check Docker and 9092. For no messages, check topic, group, offsets, and whether another consumer already owns the partition. Duplicate messages can follow a crash before auto-commit. Ordering is only per partition. Topic existence can be checked with kafka-topics --describe.

Workflow

Observe logs first; verify broker, topic, group, partition, and files.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

32. Kafka Consumer Lag

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Lag is produced records minus consumed progress. The code does not expose a lag metric, but Kafka CLI or monitoring tools can measure it. Lag grows when task processing is slower than production, especially because the simulated worker handles one message at a time.

Workflow

More input than processing capacity -> backlog -> lag.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

33. Beginner Project to Real Project

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The beginner stage is Flask -> one Kafka topic -> one worker. An intermediate system adds APIs, multiple consumer groups, and a database. Production adds multiple brokers, replication, observability, schema governance, security, and separate topics for business events.

Workflow

Scale by separating request handling, transport, workers, and storage.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

34. Real-World Use Cases

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Examples map directly to the same pattern: orders producer -> orders topic -> fulfillment consumer; payment event -> payment topic -> notification consumer; logs -> log topic -> analytics consumer. The current project specifically represents background task execution.

Workflow

Event producer -> named topic -> specialized consumer group.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

35. Scalability

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Throughput growth requires more partitions, consumers, brokers, and storage. A group can parallelize only up to the number of partitions. This project has one worker loop and no throughput or lag monitoring, so it is a learning-scale system.

Workflow

Increase partitions -> add group members -> distribute work -> monitor lag.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

36. Kafka Replication

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Replication is not meaningfully implemented here: one broker and replication factors of 1 are configured. In production, a partition has a leader and followers; ISR members replicate it, and a healthy follower can become leader after failure.

Workflow

Leader accepts writes -> followers replicate -> failover if leader fails.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

37. Retention

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Kafka does not delete a record when this consumer reads it. Retention is broker policy. This Compose file does not set application retention values. Retention makes replay possible, unlike a simple destructive queue.

Workflow

Write -> retain by time/size -> consumer reads -> record remains until retention removes it.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

38. Kafka vs Traditional Message Queues

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Kafka favors durable logs, replay, partitions, and high throughput. RabbitMQ-style queues often emphasize routing and work-queue delivery. Choose Kafka for streams and replay; choose a traditional queue for simpler command delivery when replay and partition scaling are not central.

Workflow

Compare ordering, replay, retention, consumer groups, and throughput.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

39. Security

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

No authentication, authorization, TLS, SASL, or secret manager is configured. The broker uses PLAINTEXT. Environment variables only override the bootstrap address. Production should protect credentials, encrypt traffic, restrict ACLs, and avoid secrets in source files.

Workflow

Current: trusted local network. Production: identity + encryption + ACLs.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

40. Monitoring

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The code logs connections, sends, partitions, offsets, and task results, but it does not expose broker health, consumer lag, throughput, disk, or under-replicated partition metrics. Recommended tools include Kafka UI, JMX exporters, Prometheus, and Grafana.

Workflow

Instrument producer, broker, consumer, and business outcomes.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

41. Kafka Design Best Practices

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Good next steps are stable topic names, deliberate partition counts, keys for ordering, bounded message sizes, retries, dead-letter topics, idempotent handlers, schema validation, and lag monitoring. In this project, start with atomic result writes and a real durable database.

Workflow

Design topic -> choose key/partitions -> validate -> process safely -> observe.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

42. Complete API Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Implemented API routes are GET /, POST /submit-task, GET /tasks, and GET /health on port 5001. POST accepts JSON or form data. It creates the event and returns 201; the consumer does not respond synchronously to the original POST.

Workflow

Client -> Flask -> Kafka -> worker -> files -> GET /tasks.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

43. Database Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

There is no database. submitted_tasks.json and task_results.json are local files, and queue_buffer.json is a fallback queue. This is useful for learning but unsafe for multiple processes, crashes during writes, and horizontal scaling.

Workflow

Consumer -> atomic JSON result file -> producer merges -> dashboard.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

44. Complete Architecture Diagram

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

A faithful architecture is: browser/API -> Flask producer -> JSON serializer -> Kafka broker -> task-queue -> partition -> task-processor-group -> consumer.py -> task_results.json -> Flask sync -> dashboard. The fallback branch writes queue_buffer.json when Kafka send fails.

Workflow

Normal path and fallback path are both explicit in the code.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

45. Sequence Diagram

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Sequence: client POSTs; Flask validates and creates payload; producer sends; broker assigns metadata; consumer fetches; worker marks PROCESSING; worker sleeps through three stages; worker writes COMPLETED; dashboard polls GET /tasks. The original POST does not wait for completion.

Workflow

POST -> send -> fetch -> process -> persist -> poll.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

46. Folder Structure

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The repository is small: producer.py, consumer.py, docker-compose.yml, requirements.txt, templates/dashboard.html, and JSON state files. __pycache__ is generated runtime output. README.md is existing project guidance.

Workflow

Infrastructure + producer + consumer + UI + local state.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

47. File-by-File Explanation

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

producer.py owns HTTP and publishing; consumer.py owns polling and work; docker-compose.yml owns Kafka infrastructure; dashboard.html owns browser rendering and five-second polling; requirements.txt owns Python dependencies; JSON files hold demo state.

Workflow

Entry point -> imported library -> Kafka/file side effect -> next component.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

48. Kafka Terminology Glossary

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Core terms: broker is a Kafka server; topic is a named log; partition is an ordered lane; offset is a position; producer writes; consumer reads; group shares work; retention controls lifetime; lag is unprocessed distance; serialization converts objects to bytes; rebalancing redistributes partitions.

Workflow

Use the glossary to translate Kafka words into project actions.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

49. Interview Questions

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Beginner interview answers should be project-specific: Kafka decouples Flask from slow work; task-queue is the topic; task-processor-group shares partitions; offsets track progress; JSON is the wire format; auto-commit means duplicates are possible; one broker is a single point of failure.

Workflow

Answer what this code does before describing ideal Kafka behavior.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

50. 2-Minute Project Explanation

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Two-minute answer: This project is a Flask task queue. A user submits a task; producer.py validates it, assigns an ID, serializes JSON, and sends it to task-queue. consumer.py, in task-processor-group, reads it, simulates priority-based work, and writes status to task_results.json. The dashboard polls Flask to show progress. It is a learning system, not a production cluster.

Workflow

Keep the interview story: problem -> producer -> Kafka -> consumer -> outcome.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

51. 5-Minute Technical Explanation

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Five-minute answer should add broker localhost:9092, one topic, group_id, partition/offset metadata, earliest reset, automatic commits, status transitions, JSON serializer/deserializer, retries, fallback buffer, and the limitations of local files and one broker.

Workflow

Expand the two-minute story with one message's technical lifecycle.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

52. Common Beginner Confusions

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Kafka is not the database or API. A topic is not a table; a partition is not a topic; a group is not one consumer; reading does not immediately delete a record; two different groups can read the same record; duplicates are possible; offsets enable restart and replay.

Workflow

If confused, return to: durable log + partitions + consumer progress.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

53. Documentation Files

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The requested split is represented by docs/topics/01-...md through 60-...md, plus a master document. Each topic file is beginner-oriented and names the actual project files and workflow.

Workflow

Open a topic file for focused learning; use the master for the complete path.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

54. Master Markdown File

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The master file combines all 60 sections, the project facts, diagrams, commands, workflows, limitations, and final report. It is the easiest file to search and share as source Markdown.

Workflow

Topic pages -> ordered master Markdown -> PDF rendering.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

55. PDF Document

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The PDF is a formatted study guide with title page, contents, project facts, and one topic section per numbered requirement. It includes readable workflows, code references, commands, and production notes.

Workflow

Markdown content -> ReportLab styles -> page breaks -> validated PDF.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

56. PDF Quality Check

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Quality checks include generated page count, extractable text, section presence, output file existence, and PNG rendering of representative pages. Visual inspection should confirm headings, wrapping, footer numbers, and diagrams are readable.

Workflow

Generate -> render -> inspect -> fix -> deliver.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

57. Final Kafka Flow

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The master flow is: user/application -> Flask producer -> JSON serialization -> broker -> task-queue -> partition/offset -> consumer group -> consumer -> JSON deserialization -> processing -> result file -> dashboard. The configured system has one broker and one normal worker.

Workflow

This is the complete mental model for the repository.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

58. Final Beginner Explanation

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Remember: a user submits a task to Flask. Flask puts a JSON event into Kafka. Kafka stores it in a topic partition with an offset. The consumer reads it, performs work, and records the result. The dashboard learns the result by polling Flask. Kafka makes the slow work asynchronous.

Workflow

Short memory aid: submit -> store -> read -> work -> record -> display.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

59. Final Project Report

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

Project: Apache Kafka Task Queue System. Language: Python. Library: Flask, kafka-python, Jinja2, colorama. Broker: localhost:9092, one Docker broker. Topic: task-queue. Group: task-processor-group. Producer: producer.py. Consumer: consumer.py. State: JSON files. API: four Flask routes. Missing: database, replication, security, schema registry, metrics, dead-letter topic, and true exactly-once processing.

Workflow

Recommended improvements: durable storage, multiple brokers, explicit partitions, secure listeners, monitoring, retries, idempotency, tests, and CI.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.

60. Most Important Rule: One Message

Actual project: Python Flask producer in producer.py; kafka-python KafkaProducer; consumer worker in consumer.py; topic task-queue; broker localhost:9092 by default; consumer group task-processor-group; Docker Compose runs one Confluent Platform 7.5.0 broker plus ZooKeeper on 2181; JSON values; task status files are submitted_tasks.json and task_results.json.

Beginner meaning

The deliverable set contains 60 topic Markdown files and one master Markdown file. The PDF is a single complete document so it can be printed or shared. Each topic is kept in the same order as the pasted requirements.

Workflow

Use individual files for focused study and the PDF for a linear course.

Implementation note

This page describes what the current repository implements. Any production improvement mentioned is a recommendation, not an existing feature.